

V2X 시각장애인 보행 지원 — 인수인계 및 현황 평가

2026-08-06 작업분 · TTC 연구, AI 젯슨 연동, 팀 조율 사항

0. 한 장 요약

오늘 답한 질문: "최적의 TTC 임계값은 몇 초인가"

답: 그런 값은 존재하지 않는다. 모델이 탐지한 위험의 34%가 TTC 무한대 상태였고(측면 53%, 후방 75%), 임계값을 어떤 값으로 두든 잡히지 않는다. 최적화 대상은 임계값이 아니라 판단 구조다.

항목	상태
규칙 독립 정답(오라클) 설계	완료 · PET와 96.7% 일치로 교차검증
3-way 비교 (규칙 / 고정TTC / 모델)	완료 · 모델이 +18.9%p (동일 오경보)
안전 하한 2.0초	젯슨 배포·가동 확인
AI 모델 젯슨 연동	가동 확인 (18:07) · 실제 경보 경로
논문 PDF (8쪽)	완료 · docs 및 ttc_study/
실험 계획·검증 도구	완료 · 실험은 8/8(토) 예정

테스트 344개 통과(03_jetson 255, ttc_study 89). GitHub 양쪽 저장소 반영.

1. 현재 시스템 구조

최종 위험도 = $\max(\text{안전 하한}, \text{규칙 점수표}, \text{AI 판정})$

층	담당	근거	실패 시
② 상한 게이트	규칙	멀고 접근하지 않음	—
① AI 조건부 판단	모델	학습 (오라클 라벨)	규칙이 대신 판단
안전 하한 2.0초	규칙	물리 계산 + 국제 규격	해당 없음

세 층이 서로를 덮어쓰지 않는 것이 설계의 핵심이다. 안전 하한은 AI보다 세다 — AI가 안전하다고 판단해도 TTC 2초면 레벨 3이 나간다. 모델은 학습 분포 밖에서 예고 없이 틀릴 수 있지만 하한은 물리로 계산한 값이라 그럴 일이 없다. 모델은 더하기만 한다 — 규칙이 올린 레벨을 내리지 못한다. 모델이 죽어도 규칙은 돈다 — 예외를 잡아 규칙 판정으로 넘어간다. --no-model 로 끄면 이전과 완전히 같다.

1.1 T_floor = 2.0초의 근거

구성	값	출처
GPS 갱신 주기	0.2 s	5Hz 전환 후 (미검증 — 3.2절 참조)
전송 지연	0.1 s	8/5 실측, 위험도 변경 29건 전수
인지·판단	0.3 s	측각 반응 0.12~0.18s의 1.5~2배
정지 동작	1.2 s	보행 급정지 0.84~1.21s의 보수적 끝
안전 여유	0.2 s	—
합계	2.0 s	GB/T 33577 최소 2초, NHTSA NCAP 2.0~2.4초와 일치

2. AI에 대한 평가

2.1 무엇을 신뢰할 수 있는가

항목	수준	근거
시뮬레이션 성능	높음	시나리오 1,200개 × 5시드, 적시경보 66.6%→85.5% (+18.9%p), 4/5 시드 승
안전성 구조	높음	3층 구조. 실측 재생에서 모델 0.0(안전) 판정을 하한이 뒤집어 레벨 3 발동 확인
배포 안정성	높음	순수 파이썬 추론, sklearn 대비 오차 2.22e-16. 젯슨에 설치물 없음
설명 가능성	중간	치환 중요도로 판단 근거 제시 가능. 개별 판정의 이유는 아직 미구현
실측 검증	낮음	실외 로그에서 모델이 반응한 구간 9스텝. 고속 상황 데이터 없음
학습 분포 밖 거동	미지	다중 차량, 실제 도로 기하, 5Hz GPS 조건에서의 행동 미확인

요약하면 '시뮬레이션 안에서는 검증됐고, 현장에서는 아직'이다. 그래서 AI를 실제 경보 경로에 넣어 안전 하한을 그 아래에 깔았다. AI가 학습한 적 없는 상황에서 침묵해도 하한이 대신 올린다.

2.2 이 AI가 규칙보다 나은 이유

치환 중요도에서 TTC는 4위(0.053)이고 접근속도가 1위(0.213)로 4배 높다. 결정적으로 모델이 잡은 위험의 34%는 TTC가 무한대인 상태였다. 옆으로 지나가는 차는 시선방향 접근속도가 0에 가까워 TTC가 무한대로 계산되지만 곧 보행자의 진행 경로를 가로지른다. 임계값 방식으로는 원리적으로 잡히지 않는 위험이고, 모델은 상대 위치 벡터·가속·선회율을 함께 보기 때문에 잡는다.

2.3 신뢰할 수 없는 것 — 정직하게

- ① 오경보율의 절대값은 시뮬레이터 생성 분포 기준이라 "하루 몇 번"으로 환산할 수 없다. 세 방법의 상대 비교로만 읽어야 한다.
- ② 고속 상황은 검증되지 않았다. 실외 로그의 차량 속도 중앙값이 1.46 m/s로 시뮬레이터(7.35)의 1/5이다. 안전상 제약이지만 공백은 공백이다.
- ③ 시각장애인 반응시간 문헌을 찾지 못해 일반 보행자 급정지 데이터로 대응했다. T_floor의 1.2초 항이 여기에 걸려 있다.
- ④ 다중 차량은 다루지 않았다. 현재 state_store가 차량 1대만 추적한다.

3. 코드에 대한 평가

3.1 구조

계층	파일	역할
연구 (PC)	ttc_study/ 14개 모듈	시나리오 생성, 오라클 라벨, 학습, 3-way 비교, 실험 검증
배포 (젯슨)	model_runtime.py	표준 라이브러리만으로 트리 앙상블 추론
	model_features.py	스트리밍 피쳐 (배치 계산과 값 일치 검증됨)
	model_gate.py	max 결합, 실패 격리, on/off
기존 파이프라인	step6/7/8	최소 수정 (last_states 노출, 하한, 게이트 호출)

연구 코드와 배포 코드를 분리한 것이 이 구조의 요점이다. 젯슨에는 scikit-learn도 numpy도 필요 없고, 연구 쪽이 아무리 바뀌어도 배포 쪽은 JSON 파일 하나만 갈아끼우면 된다.

3.2 검증 수준

대상	검증 방식
오라클 라벨	PET(교통공학 표준 지표)와 96.7% 일치. 규칙 모듈 import를 AST로 차단
시뮬레이터 기하학	등속 직선의 해석해와 대조 (miss_offset = 실제 최소거리)
모델 내보내기	sklearn 예측과 최대 오차 2.22e-16
스트리밍 피쳐	배치 계산과 소수점 6자리까지 일치
미래 참조 없음	뒤를 돌려 재계산해도 기존 값이 불변인지 확인
파이프라인 회귀	기존 테스트 233개 → 255개 전부 통과

3.3 알려진 위험 지점

- ① 모델 파일 버전 관리가 없다. risk_model.json에 학습 메타(시나리오 수, 시드, 임계값)는 담았지만 버전 태그가 없어, 젯슨의 파일이 어느 커밋에서 나온 것인지 추적이 어렵다. 실험 전 md5sum 기록을 권한다.
- ② 롤백 절차가 문서화되지 않았다. 실기에서 문제가 생기면 --no-model 을 서비스 파일에 추가하고 재시작해야 하는데, 이 절차가 어디에도 적혀 있지 않다.
- ③ rsu 저장소가 비공개라 젯슨에서 curl로 받을 수 없다. 반드시 AI-V2X-Cane(공개)의 rsu/v2x/03_jetson/ 경로를 써야 한다. curl -sO 는 404여도 조용히 14바이트 파일을 저장하므로 -fsSL -o 와 크기 확인이 필수다. 8/6에 이 함정으로 안전 하한이 올라가지 않은 채 배포에 성공한 것으로 오인했다.

4. 팀원 작업과의 조율 — 가장 시급

같은 날(8/6) 다른 팀원이 v3 트랜스포머 예측을 완성해 젯슨 자동 가동에 올렸다. 두 작업이 독립적으로 진행되어 같은 자리를 겨냥하고 있다.

4.1 한치되는 부분 — 설계가 같다

팀원이 제안한 통합 식이 최종 = $\max(\text{규칙}, \text{AI 3초 예측})$ 이고, 이번 작업의 구조와 동일하다. "AI는 경고를 앞당기기만 하고 끌 수는 없다"는 원칙도 같다. 두 사람이 독립적으로 같은 결론에 도달한 셈이라, 이 설계는 신뢰할 만하다.

팀원이 남긴 확인 요청 3가지에 대한 답은 이미 코드로 나와 있다.

팀원 질문	답
step6 NodeTracker.state_ahead() 재사용 가능?	가능. KinematicsPipeline.last_states 로 노출해 사용 중
실시간 루프에 초당 1회 ONNX 추론 여유?	ONNX 불필요. 순수 파이썬 추론이 1ms 미만. 기존 계산이 1.0ms
step8 히스테리시스를 AI에도 적용?	적용됨. max 결합 이후 stabilize 를 태운다

4.2 두 모델은 지금 충돌하지 않는다

먼저 확인해 둘 것: 두 모델은 서로 다른 자리에서 돌고 있다. 팀원이 step9로 들어오려던 실시간 경보 자리를 이번 작업이 채운 것이지, 현재 둘이 같은 출력을 두고 경쟁하는 상태가 아니다. 그래서 급하지 않다.

	v3 트랜스포머 (팀원)	GBM (이번 작업)
도는 자리	서버 시나리오 → 지도용 예측 (1분/개, 배치)	step8 → 지팡이 실시간 경보 (관측마다)
출력이 가는 곳	onnx_risk_level → 위험지도 API	risk → 지팡이 진동
학습 라벨	규칙 점수표의 미래값(risk_level_future3)	오라클 (규칙 독립)
입력	시퀀스 10스텝	단일 시점 + 변화율
데이터	SUMO v2 (다중 보행자·횡단보도·실제 지도)	파이썬 2체 운동학
배포 형식	ONNX (onnxruntime 필요)	JSON (표준 라이브러리만)
검증 기준	규칙 라벨 대비 정확도	오라클 대비 +18.9%p

4.3 그래도 결정해야 할 것

권고: 두 모델을 같은 오라클 라벨로 채점해 비교한다. ttc_study/evaluate.py 의 평가 틀은 모델 구현과 무관하므로, v3 ONNX를 predict 함수로 감싸면 그대로 채점할 수 있다. 규칙 라벨로 학습한 모델이 규칙 독립 정답에서 어떤 성적을 내는지가 이 비교의 핵심 정보다.

v3가 낮게 나와도 모델 탓이 아닐 수 있다. v3의 라벨은 팀 점수표에서 유도된 값이므로, 오라클 기준 성적이 낮다면 그것은 라벨의 문제다. 같은 데이터를 오라클로 다시 라벨링해 재학습하면 개선될 가능성이 높다.

가장 좋은 조합은 아마 '팀원의 데이터 + 이번 작업의 라벨'이다. SUMO v2 데이터는 다중 보행자·횡단보도·실제 지도 기하를 담고 있어 이번 작업의 2체 시뮬레이터보다 현실적이다. 반면 라벨은 오라클 쪽이 규칙과 독립적이다. 두 강점을 합치면 지금의 어느 쪽보다 낫다.

모델 형식에 대해: 트리 양상블은 JSON으로 옮겨지지만 신경망은 옮겨지지 않으므로, v3 계열을 실시간에 쓰기로 하면 ONNX가 필요하다. 젯슨에 onnxruntime은 이미 있다(v3가 그것으로 돌고 있다). 형식은 선택의 제약이 아니라는 뜻이다.

5. 역할 분담 검토

제안된 분담은 방향이 맞다. 다만 각 주제를 측정 가능한 목표로 좁히면 진척을 판정할 수 있다. 아래는 보완안이다.

5.1 중선 · 현준 — GPS 오차 잡기

왜 1순위인가: 통신 지연은 이미 0.1초로 측정됐다. 남은 병목은 GPS다. 8/5 기준 갱신 주기가 936ms(1.1Hz)였고, T_floor 2.0초는 5Hz(0.2초)를 전제로 계산한 값이다. 1Hz면 필요한 하한이 2.8초가 되어 전제가 무너진다.

측정 가능한 목표	확인 방법
GPS 갱신 200ms 부근 (5Hz 실측 확인)	watch_link.py 2행 또는 verify_experiment.py 1번
gps_valid=1 비율 개선 (차량 51.3% 무효 → ?)	raw 로그 집계
위치 산포 sigma 측정 (현재 가정 2.5m)	정지 상태 5분 로깅 후 표준편차
불가능한 이동 비율 (차량 3.21% → ?)	연속 좌표 간 속도 상한 검사

sigma가 2.5m보다 작아지면 시뮬레이터의 노이즈 파라미터도 낮춰 재학습해야 한다 — 학습과 현장의 조건을 맞추기 위해서다.

5.2 현준 · 민서 — AI 모듈 활용 방안 (안전성·정확성)

안전성은 이미 답이 나와 있다: 3층 구조로 AI 실패가 격리된다. 정확성도 시뮬레이션에서는 답이 나왔다: +18.9%p. 그러므로 이 주제의 남은 실질은 두 모델 통합 결정과 실측 검증이다.

측정 가능한 목표	판정 기준
두 모델을 같은 오라클로 비교	evaluate.py 결과표 1장
채택 모델 결정 및 단일화	젯슨에 하나만 남는다
실측 고속 위험 30건 확보	verify_experiment.py 3번 항목
새도우 비교: 실측에서 AI가 규칙을 이기는가	level_source 열 집계
개별 판정 근거 출력 (설명가능성)	"왜 울렸나"를 한 줄로 답할 수 있는가

5.3 민서 · 채린 · 현준 — 홈페이지 시각화

위험지도 API는 이미 가동 중이다. 차별점은 "AI가 왜 그렇게 판단했는지"를 보여주는 것이다. 위험 지점만 찍는 지도는 흔하지만, 판단 근거를 나란히 보여주는 것은 드물고 이번 연구의 결과가 그것을 가능하게 한다.

보여줄 것	왜 효과적인가
규칙 vs AI 판정 비교 (같은 궤적)	AI를 쓴 이유가 화면에서 드러난다
TTC가 무한대인데 AI가 잡은 사례	연구의 핵심 주장을 한 장면으로 설명
안전 하한 발동 구간 표시	"AI를 못 믿을 때"에 대한 답을 시각화
3초 선행 예측 (팀원 v3 뷰어)	이미 만들어져 있음 — 지도와 나란히
상황별 경보 시점 분포	alarm_ttc_by_situation.png 를 그대로 활용

주의: 팀원 문서에 따르면 onnx_risk_level의 의미가 '현재 위험'에서 '3초 내 예상 위험'으로 바뀌었다. 지도 범례와 설명 문구를 반드시 수정해야 한다.

6. 제안에서 빠진 것

빠진 항목	왜 필요한가	제안
실험 실시 담당	8/8 실험이 모든 실측 검증의 전제. 계획서는 있으나 실행 주체가 정해지지 않음	하드웨어 담당 + 현준
모델 단일화 결정	두 시가 같은 자리를 겨냥. 방치하면 젯슨에 둘이 공존하거나 충돌	현준 + 민서, 8/9까지
발표·제출 자료	팀원 문서에 8/31 제출 명시. 지금까지 결과를 엮을 사람 필요	전원, 8월 하순
롤백 절차	실기에서 AI가 문제를 일으켰을 때 되돌리는 방법이 문서화되지 않음	현준, 실험 전

6.1 일정

시점	일	담당
8/8 (토)	실측 실험 — GPS 5Hz 확인, 고속 위험 30건, 새도우 데이터	실험팀
8/9	두 모델 오라클 비교 → 단일화 결정	현준·민서
8/10~12	채택 모델 재학습(필요 시) · GPS 개선 반영	현준·민서·중선
8/13~16	홈페이지 시각화 · 발표 자료 초안	민서·채린·현준
8/17	통합 테스트	전원
8월 하순	제출 자료 (8/31 제출)	전원

7. 다음 작업 세션을 위한 시작점

새 대화에서 맥락을 복원하려면 이 순서로 읽으면 된다.

1. ttc_study/SPEC.md — 설계와 결과 전체 (12개 절)
2. ttc_study/TTC_연구_논문.pdf — 근거와 실험 (8쪽)
3. ttc_study/EXPERIMENT_PLAN.md — 실험 절차
4. 이 문서 4절 — 팀원 작업과의 조율 사항

바로 실행 가능한 명령

```
cd 03_jetson/ttc_study
python evaluate.py --n-scenarios 1200 --repeats 5 --lead-s 2.0 # 3-way 비교
python verify_experiment.py --log ../logs/<실험로그> # 실험 검증
python model_runtime.py --n-scenarios 1200 --out ../risk_model.json # 재학습
python -m pytest # 테스트 89개
```

젯슨 배포 (반드시 공개 저장소 경로)

```
B="https://raw.githubusercontent.com/joon722/AI-V2X-Cane/main/rsu/v2x/03_jetson"
curl -fsSL -o <파일> "$B/<파일>" # -sO 금지: 404를 조용히 저장한다
```